

BeatCanvas

Complete design, and possible future enhancements

A desktop application that turns a folder of photographs and one piece of music into a beat-synchronised video. It listens to the track, works out its tempo, metre, repeating rhythmic figures and structure, decides what the picture should do about each, and renders the result. This document is written so the application could be rebuilt from it alone: every constant that matters, every decision with the reasoning behind it, and every mistake worth not repeating.

Describes the state of the code on 30 August 2026.

1. What it is for

The problem is not "put photos to music"; any editor does that. The problem is that such a video almost always feels arbitrary. Cuts land near beats rather than on them, one transition repeats until invisible, and when the music changes the picture does not.

Three specific promises:

- ? **A cut lands on a beat, not near one.** Every change sits on a grid derived from the music, offset slightly early so it reads as anticipation rather than lateness.
- ? **A repeating groove gets a repeating answer.** If the music does the same thing twice, the picture does too. This is the most important promise and the hardest to keep.
- ? **No photo passes unseen.** Every picture is fully visible, uninterrupted, for a minimum span. A reveal still finishing when the next cut arrives is a bug, not a style.

Everything is local. No account, no upload; the only traffic is between the desktop window and a server on the loopback interface.

2. Constraints that shaped every decision

These are not incidental. Most unusual choices below follow from them.

- ? **Python has numpy, OpenCV, FastAPI and uvicorn, and nothing else.** No librosa, scipy, madmom, torch or demucs. Every piece of signal processing is written by hand against numpy. This is why harmonic/percussive separation is a median filter rather than a learned model, and why structure comes from a self-similarity matrix rather than a segmenter.
- ? **The machine is modest.** Rendering is CPU-only through OpenCV, about 20 ms per 1080x1920 frame, so twenty seconds of video takes a little over two minutes. Every effect must be expressible as an affine transform plus a cheap mask.
- ? **PowerShell refuses to run .ps1 scripts on the target machine.** This defeats npm and npx quietly: the command returns instantly having done nothing. Anything needing Node goes through `cmd /c npm.cmd`` or calls the executable directly.
- ? **The console is code page 1252.** Diagnostics are ASCII only, and prints that might carry

other characters are wrapped against `UnicodeEncodeError`.

The general lesson: prefer a technique that can be verified on this machine today over one that would be better with libraries that are not installed.

3. Architecture

3.1 Module map

beatcanvas/	
analysis.py	listening: tempo, metre, grid, sections, curves, reading presets
rhythm.py	repetition: bar signatures, loop detection, figures, treatments
choreography.py	direction: which treatment, when, at what pace
effects.py	the shared vocabulary, and the animation styles
renderer.py	drawing: one frame at a time, three layers of motion
animations.py	the in-slide movement library
template.py	the data model: Template and Clip
photos.py	loading, EXIF orientation, cover fit, caching
beats.py	audio loading, STFT magnitude, tempo seed, peak picking
explain.py	the written account of a track
store.py	job store, SQLite
worker.py	job queue: prepare, choreograph, render
app.py	FastAPI: pages, API, form handling
dialogs.py	native file and folder pickers
config.py	paths, tool discovery, frame sizes
capcut.py	export an edit as a CapCut draft
web/templates/	index.html, the whole interface
desktop/	
main.js	Electron main process: owns the port and the Python lifecycle
preload.js	the narrow bridge exposed to the page
launch.bat	starts Electron without going through a blocked shell
tools/	verification scripts and utilities

3.2 The flow of one render

```
music file -> beats.load_audio -> STFT magnitude

analysis.analyse  <- reading preset

MusicAnalysis: grid, sections, bursts, drops, curves

rhythm.analyse  <- animation style

RhythmMap: figures, loop period, one Treatment each

choreography.compose

Plan: a Template whose Clips carry all three motion layers

renderer.render  <- photos.PhotoSet

mp4 with audio, plus a written account in _logs/
```

3.3 The one structural decision worth stating

The choreographer emits a **Template** whose Clips carry the choreography, rather than a separate composition model. A Clip was extended with the fields the three motion layers need. This reuses the renderer, the job queue, the adaptation logic that fits an edit to however many photos exist, and the whole interface. A parallel model would have doubled all of it for no gain.

4. The data model

4.1 Clip

One photo's time on screen, and everything that happens to it.

start, end	float seconds
slot	int, which photo (modulo how many exist)
animation	str, in-slide movement from animations.LIBRARY
intensity	float, scales the movement
offset_x, offset_y	float, position in HALF-CANVAS-HEIGHTS (see 11.4)
cover_mode	str, how the picture is fitted
reveal_kind	str, the arrival: how the photo appears
reveal_duration	float seconds
reveal_at	list[float], beat times at which pieces land
reveal_total	int, how many pieces
reveal_shown	int, how many have landed
reveal_seed	int, randomises piece order deterministically
reveal_order	"sequence" "random"
reveal_shape	"box" "circle"
transition	str, how the previous photo leaves
transition_duration	float seconds
micro	tuple[str], beat reactions active
micro_beats	list[float], beat times to react on
micro_intensity	float
focal_x, focal_y	float, the point the movement pulls toward
figure	int, which rhythmic figure chose this, or -1
section	str, the section label it falls in

4.2 Template

``width`, `height`, `fps`, `clips`, `audio_path`, `audio_name`, `reveal_tiles`,
`reveal_order`, `reveal_shape`, `min_clip_duration` (0.20 s; below that a cut reads as a flicker), `max_seconds`, `fill`, and `notes`` -- plain-language statements about why the edit is what it is, stored on the job so a result can be traced.

``Template.adapt(photo_count, fill)`` fits the edit to however many photos exist; ``fill`` is ``repeat`, `truncate` or `stretch``.

4.3 Frame sizes

`portrait` 1080x1920, `landscape` 1920x1080, `square` 1080x1080. Motion is expressed relative to height and divided by aspect for horizontal movement, so a gesture travels the same visual distance whatever the shape.

5. The analysis engine

`analysis.analyse(path, duration, fps, sensitivity, preset) -> MusicAnalysis`

5.1 Front end

- ? Audio decoded to mono at **22050 Hz**.
- ? STFT frame **2048**, hop **512**: about 43 frames per second.
- ? Spectral flux is the half-wave-rectified first difference of magnitude, summed over bins.

Three flux signals, because they answer different questions:

- ? **overall**, from the full magnitude, for tempo and the grid.
- ? **percussive**, for onsets marking hits.
- ? **low band**, bins below **150 Hz**, for where a bar starts. Bass is where the downbeat lives; the full spectrum lets a bright off-beat outvote a kick.

5.2 Harmonic and percussive separation

A median filter over a log-spaced decomposition of **160 bands**. A median across time keeps what is steady, which is harmonic; a median across frequency keeps what is broadband and brief, which is percussive. Full spectral resolution was too slow and the result was indistinguishable.

This is a proxy for stem separation and the code says so. The **voice** attributed to a moment -- drums, bass, melody, ambient -- comes from which band dominates and whether the energy is percussive or held. It is not an instrument classifier.

5.3 Tempo and metre, decided together

`estimate\_grid(flux, low\_flux, seed\_bpm, duration) -> (bpm, meter, phase, offset)`

The most important function in the application, and its shape is the result of a failure. The original estimated tempo first and metre second. That produces a specific, confident, wrong answer: a four-four song read as three-four at one and a half times the speed. Three of the wrong beats occupy exactly the span of two right ones, so every third detected bar line is real and the reading scores well.

The fix is joint estimation. Candidate tempos are generated around the autocorrelation seed including its half, double, two-thirds and three-halves. Each is scored **paired with each candidate metre**, by a comb filter over the flux, with the low band weighted double so bar-start evidence counts twice. A pairing is accepted only if both halves are explained, so a tempo justifiable only by an implausible metre is ruled out.

titanium-170190.mp3	98.1 BPM 3/4	->	87.2 BPM 4/4	(2.75 s bar)	correct
bombinsound	96.9 BPM 3/4	->	129.2 BPM 4/4		correct

The third case is unresolved; see section 19.

5.4 The grid

``build_grid`` lays a regular grid at the chosen tempo and pulls each beat onto the nearest real onset within a tolerance. Snapping can add or drop a beat at the edges, so ``find_downbeats`` runs **again** against the grid actually built rather than trusting the phase found on the ideal grid.

``find_downbeats(times, low_flux, flux, meter_options) -> (meter, phase)``. Passing a single-element ``meter_options`` forces the metre and finds only the phase, which is how a reading preset pins a metre.

Each beat becomes a ``GridBeat``: ``time``, ``beat_in_bar``, ``bar``, ``strength``, ``voice``, ``energy``, ``is_downbeat``.

5.5 Energy curves

Band envelopes plus brightness and loudness, resampled to the **video frame rate** so they can be indexed by frame number rather than interpolated at draw time.

bass	20-150 Hz	attack 0.010 s	release 0.200 s	
mid	150-4000 Hz	attack 0.030 s	release 0.300 s	
high	4000-11025 Hz	attack 0.005 s	release 0.080 s	
brightness	spectral centroid	attack 0.050 s	release 0.400 s	
energy	RMS loudness	attack 0.020 s	release 0.350 s	gamma 1.2

``condition(curve, fps, attack, release, gamma)`` applies an asymmetric envelope follower. The published constants for this shaping are in milliseconds and assume a much higher frame rate; at 30 fps a 2 ms attack is under a tenth of a frame and strobos. **Attack is floored at 2.5 frames.** That single clamp is the difference between a reaction that is felt and a flicker that is seen.

5.6 Structure

A self-similarity matrix over the band decomposition, then **Footnote checkerboard novelty** for boundaries, snapped to bar lines. Any block longer than eight bars is split, because one fifteen-bar "verse" gives the choreographer nothing to work with. Blocks are labelled from energy and similarity group: intro, verse, build, chorus, drop, breakdown, bridge, outro.

These labels are a best guess from energy and repetition, and the written account says so plainly rather than implying a trained model produced them.

5.7 Bursts and drops

? ``find_bursts`` locates runs of closely spaced hits -- fills. Five hits inside one beat is the case the piece-by-piece reveal exists for.

? ``find_drops`` requires a sustained rise held at least one second, so a single loud transient is not mistaken for a structural moment.

5.8 The Event Timeline

``MusicAnalysis.to_dict()`` emits the whole analysis as plain data: grid, sections, bursts, onsets, drops, downsampled curves. This is the contract between listening and everything downstream, and it is what makes the analysis testable in isolation.

6. Reading presets

Tempo and metre are estimates, and an estimate can be confidently wrong in a way no amount of threshold tuning fixes. Three progressively stronger three-four penalties were tried on the failing track -- 0.78, 0.62, 0.45 -- and it survived all three, because the wrong reading genuinely fits the available evidence better. Continuing would have been overfitting to two tracks.

So the application offers **several deliberately different readings**, and a wrong one is stepped around in a second. A preset carries both halves of a reading: what the tempo and metre are taken to be, and how much difference between two bars counts as a different figure. The second half is what decides whether you see one animation or twelve.

Preset fields

<code>tempo_factor</code>	multiply the chosen tempo; 0 leaves it alone
<code>meter</code>	force the metre; 0 decides from the music
<code>subdivisions</code>	slots per beat when comparing bars
<code>same_figure</code>	how alike two bars must be to count as one figure
<code>use_loop</code>	look for a loop and group by position within it
<code>cohere</code>	whether every figure shares one entrance

preset	factor	metre	subdiv	alike	cohere	for
auto	-	-	4	0.86	yes	the default
one-groove	-	-	2	0.55	yes	a loop; maximum consistency
four-four	-	4	4	0.86	yes	bar looks half or a third too short
half-time	0.5	4	4	0.86	yes	edit feels twice as busy as the song
double-time	2.0	4	4	0.86	yes	edit feels sluggish
fine-detail	-	-	8	0.93	NO	every fill gets its own treatment
waltz	-	3	4	0.86	yes	a genuine waltz or 6/8
two-thirds	0.667	4	4	0.86	yes	a four-four song read as three-four

``two-thirds`` exists because neither halving nor forcing four-four lands on the right bar when the misreading is a 3:2 error. Halving overshoots; two thirds is exact.

Little Do You Know Beat Cry

auto	110.7 BPM	3/4	bar 1.63 s	2 figures	
four-four	110.7 BPM	4/4	bar 2.17 s	3 figures	
half-time	55.4 BPM	4/4	bar 4.33 s	8 figures	
two-thirds	73.8 BPM	4/4	bar 3.25 s	1 figure	<- the measured truth

7. The rhythm engine

``rhythm.analyse(music, subdivisions, threshold, use_loop, cohere, look)`` -> RhythmMap`

7.1 Bar signatures

Each bar becomes a vector of `meter x subdivisions` slots holding the onset strength within each slot. Vectors are normalised, so comparison is by shape rather than by loudness.

7.2 Loop detection first

`find\_loop(matrix) -> (period\_in\_bars, strength)` autocorrelates the sequence of bar signatures. On a looped track, **position within the loop is a far better guide to what a bar is than comparing it with its neighbours**, because a slightly wrong grid makes adjacent bars look different when they are not.

`group\_by\_loop(matrix, period, threshold)` then groups bars by loop position. The effect on the reference loop was decisive: three figures collapsed to one, and the rate at which the animation changed fell from about fifty per cent of cuts to six. Without a loop, `group(matrix, threshold)` compares bars pairwise.

7.3 Figures

Each group becomes a `Pattern`: `id`, blended `signature`, the `bars` and `starts` where it occurs, its `voice`, its `energy`, and measured character:

- ? **density** -- how many slots are occupied.
- ? **syncopation** -- how much energy falls off the beat.
- ? **longest run** -- the longest chain of consecutive occupied slots, which identifies a fill.
- ? **accent ratio** -- how much louder the strongest hit is than the mean.

Figures are renumbered by first appearance, so figure 0 is what the song opens with.

7.4 Treatment

`choose(pattern) -> Treatment` maps character to response. The rule is that **treatment is a function of the musical situation, not of a desire for variety**. An earlier version picked from a pool while refusing immediate repeats; the result was an edit that changed its mind constantly, which was the first thing complained about.

a fill (long run of hits)	-> pieces, one piece per hit
syncopated (energy off-beat)	-> the picture travels rather than settles
sparse and heavy	-> weight: slam, shake, throb
dense and even	-> a steady groove; open arrival, gentle movement

`Treatment` carries `reveal`, `movement`, `transition`, `micro`, an explicit `gentle` override, and `because` -- a sentence naming the evidence, which appears in the written account so the decision can be argued with.

Each figure also has a **softened counterpart** for quiet passages, via `GENTLE\_ARRIVAL` and `GENTLE\_MOVEMENT`. A figure is therefore allowed exactly two arrivals across the video -- its own and the softened form -- plus a plain cut where a photo was too short to build anything. Anything beyond those three means something other than the figure is choosing, and the guard treats it as a failure.

7.5 Cohesion

The fix for the complaint that mattered most, and the second attempt at it.

Giving every figure its own arrival is correct in principle and wrong in effect. Two figures alternating bar by bar change the entrance on every second photo. A viewer does not see "figure A, figure B"; they see an edit that cannot make up its mind. Measured, `auto` was changing the arrival on **52 per cent** of cuts and `four-four` on **100 per cent**.

So the figure covering the most bars sets the arrival and every other figure adopts it, with the substitution recorded in that figure's `because` text. What still separates the figures is movement inside the frame and beat reactions, which are felt rather than noticed.

The first version gated this on loop detection. That failed instructively: on a twenty-second window the loop was not detected, so four different entrances came back. **The detector being uncertain is not a reason to make the video restless.** Cohesion is now unconditional and controlled by the preset instead.

churn before	after		
auto	52%	0%	
four-four	100%	0%	
double-time	60%	0%	
half-time	62%	14%	
fine-detail	52%	52%	(deliberate)

8. Animation styles

A style, a `Look` in the code, is a **separate axis from the reading**. The reading decides when something happens; the style decides what. Keeping them apart means circles can be asked for over a waltz reading without one choice overruling the other.

A style **narrows, never adds**. Every name it can use already exists in the verified vocabulary, so no style can request something the renderer has not been proven to draw.

style	arrivals permitted	shape	transitions
mix	everything	music	choreographer
premium	pieces, circle_wipe, zoom_unveil, shutter, soft_focus	box	dissolve, whip, cut
grids	pieces	box	cut, dissolve
circles	pieces, circle_wipe	circle	dissolve, cut
wipes	shutter, split, diagonal_wipe, circle_wipe	-	whip, cut
zooms	zoom_unveil, zoom_burst, scale_pop	-	flash, cut
soft	fade, soft_focus, circle_wipe	-	dissolve
punch	cut, scale_pop, slide_in, zoom_burst	-	whip, flash, cut

Two details make this work rather than merely appear to:

? **Substitution is by family, not list position.** `nearest\_reveal` prefers an allowed arrival of the same character -- percussive, melodic or heavy -- as the one the figure earned. Otherwise a figure that earned a hard entrance is handed a two-beat crossfade purely because it was first in the list.

? **The softened form is pinned too.** The default gentle form of a grid build is a circle, which is not a grid. Without pinning, a quiet passage becomes the one place a different kind of arrival appears.

Piece shape and transitions are not part of a figure's treatment, so a style applies those to the finished edit, clip by clip, in the worker.

9. The effects vocabulary

The catalogue this grew from lists some sixty effects. That is a menu, not a requirement, and it says as much: five well-timed effects beat sixty badly-timed ones. What follows is a smaller set chosen so every section palette has something sharp for drums, something smooth for held notes, something heavy for bass, and enough range that no palette must repeat.

Each effect carries a **family** -- percussive, melodic or heavy -- and a duration in **beats**, so its length follows the tempo rather than being fixed in seconds. A percussive arrival happens inside half a beat; a melodic one breathes over two.

9.1 Arrivals: how a photo appears (12)

cut	0.0 beats	percussive	instant, the hardest possible accent
scale_pop	0.5	percussive	snaps in oversized, settles with overshoot
pieces	1.0	percussive	arrives a piece at a time, one per beat
shutter	0.6	percussive	opens like blinds, strips staggered
split	0.6	percussive	the frame parts down the middle
slide_in	0.75	percussive	travels in from one side and stops
zoom_burst	0.75	heavy	slams out of a close-up with a shake
diagonal_wipe	1.2	melodic	an angled edge sweeps across
circle_wipe	1.5	melodic	a circle opens outward from the subject
fade	2.0	melodic	crosses over from the previous photo
soft_focus	2.0	melodic	starts far out of focus and sharpens
zoom_unveil	2.5	melodic	opens on a detail, pulls back to the whole

9.2 Transitions: how a photo leaves (4)

Kept short on purpose. Most cuts should simply be cuts, and what happens on the beat either side matters far more than what happens between.

cut	0.0 beats	percussive	no transition at all
flash	0.3	heavy	a bright flare across the cut, biggest moments only
whip	0.4	percussive	a fast smeared swipe, the photo blurring as it goes
dissolve	1.0	melodic	opacity blend into the next photo

9.3 In-slide movement (10)

Continuous motion while a photo is on screen. Names are those of `animations.LIBRARY`, because that is what the renderer already drives.

Zoom In	melodic	slow push toward the subject
Zoom Out	melodic	slow pull back to the whole frame
Drift	melodic	gentle glide with a little zoom
Glide	melodic	steady travel that never settles
Sway	melodic	slow lean one way and back
Pulse	percussive	breathes with the beat
Bounce	percussive	springs on arrival
Spin	percussive	turns slowly while zooming
Punch In	percussive	hard scale snap, then still
Cut	percussive	no movement at all

9.4 Beat reactions (6)

Every amount is small and the choreographer scales them down further. These should be felt rather than seen, and the guard enforces it: a reaction must change the frame measurably and must stay under a ceiling. Measured deviations out of 255:

zoom_pulse	a few per cent of zoom punched in on the beat	2.77
brightness_flash	a brief lift in brightness	20.95
vignette_pulse	edges darken and release with the beat	6.42
shake	a short decaying camera shake on heavy hits	2.67
chromatic	colour channels part briefly on bass hits	1.33
bass_throb	a continuous swell following the low end	2.66

10. The choreography engine

``choreography.compose(music, pace, width, height, fps, max_seconds, rhythm_map) -> Plan``

10.1 Pace

pace	bars per photo	micro intensity
slow	4.0	0.30
medium	2.0	0.60
fast	1.0	0.90
accurate	0.5	1.00

10.2 Section palettes

Each section label has a palette: permitted arrivals, transitions, in-slide movements, beat reactions, an intensity and a hold factor. An intro is patient and soft; a drop is hard and heavy; a breakdown withdraws.

On a **looped** track, section softening is switched off (``calm=False``, ``hold=1.0``). A loop has one character throughout, and letting section labels vary the treatment reintroduces exactly the restlessness cohesion removes.

10.3 Casting rather than variety

``_Casting`` selects within a palette as a function of the musical situation. Variety appears only as an accent, roughly every eighth clip. The earlier anti-repeat random picker is the direct cause of the churn in 7.5 and was removed.

10.4 Two constants that matter more than they look

? `MIN_FULL_VIEW = 0.40` -- every photo must be completely visible, with nothing arriving or leaving, for at least forty per cent of its time on screen. ``_enforce_full_view()`` shortens arrivals and transitions until this holds. Measured: 0.53 s and 0.54 s of clear view on the reference edits. This is the promise that a photo is never gone before it was noticed.

? `EARLY_OFFSET = 0.055` -- every cut moves 55 ms earlier than the beat. Perception treats a change slightly ahead of a transient as landing on it; exactly on the sample reads as

late. The first clip is forced to 0.0, because otherwise the video opens on 55 ms of black, which looks like a fault.

10.5 Bursts add, never force

A burst adds `pieces` to the candidate pool for the clips it covers and makes it preferred. It never forces it. Forcing was tried and made every clip identical, a worse failure than missing a fill. Calm sections are excluded entirely.

10.6 Output

`Plan` carries the `template`, the `pace`, an `explain` list -- one line per clip saying what it got and why -- and `notes`. All of the reasoning is kept, not the first few lines, because a complaint about the result is otherwise impossible to trace.

11. The renderer

Each frame is one photo under an affine transform, optionally smeared by a directional blur, composited onto the canvas. Cheap enough on the CPU that a browser-based renderer has no advantage.

11.1 Frame composition

`render_frame(template, photos, time_s, canvas):`

1. Find the clip at `time\_s`.
2. Compute its `Arrival`.
3. Compose the clip: photo, in-slide animation state, geometric reactions, arrival shape.
4. If the arrival needs something underneath -- a wipe, a dissolve -- compose the *previous* clip at its last moment and blend through the arrival mask.
5. Apply non-geometric reactions and any flare across the cut.

11.2 Arrival, and the single exit

`_arrival()` returns an `Arrival`: `scale`, `offset\_x`, `offset\_y`, `alpha`, `blur`, `smear`, `flash`, `mask`.

****Every path through `\_arrival` exits through `\_cover\_travel`.** This is not tidiness. An earlier version had early returns for some arrival kinds, which skipped the compensation that guarantees the frame stays covered and left 28 per cent of the frame black during a whip handover. One exit, always compensated.

`_wipe_mask()` builds the masks for shutter, split, circle_wipe and diagonal_wipe.

11.3 Piece reveals

`pieces_shown(clip, time_s)` reads the count off `reveal\_at` -- the actual beat times -- rather than interpolating over the clip. The earlier version drew the complete photo from frame one. Unrevealed area is darkened to 14 per cent rather than blacked, so the frame is

never empty, and the piece that has just landed gets a brief 45/255 lift so the eye is drawn to the beat.

Measured: five hits uncover 19, 39, 58, 78 and 100 per cent of the frame.

11.4 Offsets are in half-canvas-heights

Position offsets use CapCut's normalised space, where one unit is half the canvas height. A horizontal offset must therefore be **divided by the aspect ratio** before compensation. On a 1080x1920 frame the axes differ by nearly a factor of two, and treating them alike undercompensates enough to leave a visible black strip down one side.

11.5 Headroom

``required_headroom(template)`` computes how large each photo must be loaded, as a multiple of the canvas, given the largest on-screen scale any clip reaches including arrival zooms. Photos are loaded oversized once and cached in a ``PhotoSet``; a clip is on screen for many frames and re-decoding per frame would dominate render time.

11.6 Cost

17-25 ms per frame at 1080x1920 on the target machine. A twenty-second edit took 138 seconds end to end including audio muxing.

12. Jobs, queue and storage

SQLite via ``store.py``. A ``Job`` holds name, options, stage, progress, output path, notes and error. ``worker.py`` runs one job at a time in a thread, updating stage and progress so the page can show what is happening: **listening to the music**, **designing the edit**, **rendering**.

Two modes:

? **Automatic** -- ``_choreograph()`` runs analysis, rhythm and choreography. The reading preset and the animation style are read from the job options here, and the worker builds the rhythm map itself so both halves of the preset reach the figure finding.

? **Template** -- ``_fit_template()`` cuts a fixed style to the supplied track. Hand-edited beat markers, if present, win outright over detection.

``0`` is a legitimate value for ``max_seconds``, meaning the whole track, and must be distinguished from absent. Treating it as falsy is what previously capped every render at thirty seconds.

Every render writes the full account of the track to ``_logs/tune_<name>_<preset>.txt``. A video that feels wrong is hard to reason about; a page describing what was heard is not.

13. HTTP API

GET /

the page

GET /api/health	readiness, used by the Electron shell
POST /api/jobs	create a job from the form
GET /api/jobs	recent jobs with stage and progress
GET /api/jobs/{id}	one job
POST /api/analyse	beat preview for the marker editor
POST /api/browse	native file or folder picker
POST /api/open-folder	reveal a path in the file manager
POST /api/drop	accept dropped photo files
POST /api/reveal	show one output file
GET /api/preview	a still frame for the current settings

Form fields of note: ``template``, ``name``, ``music_path``, ``photo_folder``, ``photo_order``, ``frame``, ``pace``, ``analysis`` (the reading), ``look`` (the animation style), ``max_seconds``, ``sensitivity``, ``fill``, ``direction``, ``intensity``, ``cover_mode``, ``reveal_tiles``, ``reveal_shape``, ``beat_markers``, ``output_dir``.

The ``analysis`` field arrives under an alias, because a parameter of that name would shadow the ``analysis`` module inside the handler.

14. The interface

One page, stepped: choose a style, choose music, choose photos, adjust, make the video, watch the queue. Controls are grouped by which style needs them -- ``autoOnly``, ``moveOnly``, ``revealOnly``, ``anyOnly`` -- and every group starts **disabled**, so a default value for an option the chosen style does not use is never submitted.

Each dropdown explains itself in a line beneath it, populated from the same table the engine reads. A name like "one-groove" or "premium" means nothing until you have tried all of them; the description is what makes the choice a choice.

14.1 Remembering what was chosen

Every control in ``_optionFields`` is persisted to ``localStorage`` under ``bf_opt_<id>``, restored after the template's own defaults are applied, and saved both on submit and on change.

Three things learned here:

? A control missing from that list **silently snaps back to its default** on the reload after submit. That is worse than not remembering at all, because the form looks like it kept the choice. ``pace``, ``analysis`` and ``look`` were all missing -- the three decisions that most change the result were the three that reset.

? Restoring a value in code does **not** fire the change event, so the description beneath a dropdown must be refreshed explicitly. Otherwise the select reads "Grids only" while the text beneath still describes "Mix".

? Saving only on submit loses the choice if something is changed and then reloaded, or if a second window is opened.

14.2 Native pickers

``dialogs.py`` opens real file and folder dialogs through tkinter, exposed at ``/api/browse``, with ``/api/open-folder`` to reveal results in the file manager. It is kept as a fallback so the page works identically in a browser and inside the desktop shell.

15. The desktop shell

Electron 38.4.0. `desktop/main.js` is the main process and it owns two things deliberately.

****It owns the engine's life.**** Python is started when the window opens and stopped when it closes, killing the whole process tree so uvicorn and any ffmpeg beneath it go too. Started separately, the two drift apart: an old server keeps holding the port and the window talks to code that is no longer on disk. That happened repeatedly during development and is confusing every time. A stale process was still the cause of a change appearing not to work as late as the last session.

****It owns the port.**** A free port is obtained by listening on 0 and reading back the assigned number, then handed to Python through `BEATCANVAS\_PORT`. `run.py` uses exactly that port rather than searching from a default, because the shell is already waiting on it and quietly moving to the next would leave it waiting for ever. `BEATCANVAS\_NO\_BROWSER=1` stops Python opening a browser tab as a second copy of the same thing. The shell polls `/api/health` until it answers, so the window never shows a connection error.

```
verified 30 August 2026
starting ...GoldForge\.venv\Scripts\python.exe run.py on port 54505
engine ready on 54505          -- 2.5 seconds after launch
```

Launching must avoid PowerShell. `desktop/launch.bat` calls
`node\_modules\electron\dist\electron.exe` directly, and the desktop shortcut points at the same executable with the `desktop` folder as its argument. `npm start` works only from a shell that allows scripts.

16. Verification

Every promise in section 1 has a script that fails when it is broken. These are guards, not unit tests: they measure the rendered frames.

tools/check_deps.py	what is actually installed
tools/check_analysis.py	the analysis against a real track
tools/check_rhythm.py	figures and loop detection
tools/check_choreography.py	25 checks over paces and palettes
tools/check_layers.py	the renderer, frame by frame
tools/check_presets.py	readings, cohesion contract, style vocabulary
tools/check_page.py	what the running server is actually serving
tools/check_coherence.py	a whole edit read back
tools/explain_tune.py	the written account, plus the loop assertion
tools/make_sample.py	a watchable render through the app's own path
tools/make_shortcut.py	the desktop launcher
tools/make_icon.py	the icon

What `check\_layers` measures, and the numbers it produced:

- ? every arrival visibly changes the frame just after the cut (3.2 to 37.2 of 255)
- ? every arrival has finished by the end of its clip (residual 0.0 to 0.6)
- ? ****no arrival ever shows black**** (worst 0.0 per cent across all twelve)
- ? every wipe starts from the outgoing photo rather than emptiness

- ? every reaction changes the frame, and none exceeds the ceiling
- ? a quiet section reacts less than a loud one (1.56 at 0.15 against 2.77 at 1.0)
- ? a piece reveal lands one piece per hit (19, 39, 58, 78, 100 per cent)
- ? all four paces render with audio and show no black frame after the opening

What ``check_presets`` asserts:

- ? the app still imports with the reading wired in
- ? every preset is reachable and describes itself in more than a token sentence
- ? the readings **genuinely disagree**: at least four distinct results, because a dropdown of eight names where five produce identical numbers is an illusion of choice
- ? every preset claiming to cohere holds one entrance (under 20 per cent churn), and ``fine-detail``, which claims variety, does not
- ? **every style stays inside its own vocabulary**: all clips checked, not a sample, including softened forms, piece shapes and transitions

``check_page`` fetches the page from the running server and compares it against a list of expected markers, reporting separately on what the server sends and what the file on disk contains. It exists because a stale process holding the port is the usual reason a change does not appear, and it names that possibility in its own output.

17. Environment and setup

```
Python      C:\Users\prash\Kiro Projects\GoldForge\.venv\Scripts\python.exe
numpy, opencv-python, fastapi, uvicorn
Node        v24.19.0, npm 11.17.0
Electron    38.4.0, pinned ^38.0.0
ffmpeg      discovered via config._find_tool, overridable by BEATCANVAS_FFMPEG
port        8772 standalone; a free port chosen by the shell when run from the window
```

```
cd "C:\Users\prash\Kiro Projects\BeatCanvas\desktop"
cmd /c npm.cmd install
cd ..
tools\run.bat tools\make_shortcut.py
```

Use ``cmd /c npm.cmd``, not plain ``npm``. Environment overrides: ``BEATCANVAS_PORT``, ``BEATCANVAS_NO_BROWSER``, ``BEATCANVAS_OUTPUT_DIR``, ``BEATCANVAS_TEMPLATE_DIR``, ``BEATCANVAS_CAPCUT_ROOT``, ``BEATCANVAS_FFMPEG``.

18. Relationship to BeatForge

BeatCanvas is a fork of BeatForge, taken so the original could keep working untouched while the analysis was rebuilt. The fork renamed the package, moved the port from 8771 to 8772, and was performed by ``tools/fork_project.py`` so it is repeatable. BeatForge retains the template-driven approach; everything in sections 5 to 8 is new in BeatCanvas.

19. Known limitations

? **One track still reads wrong.** **Little Do You Know Beat Cry** is heard as 110.7 BPM in three-four; the truth is about 73.8 in four-four with a 3.25 s bar. The wrong bar is exactly two thirds of the right one, so alternate detected bar lines are real downbeats and the wrong reading scores 0.603 against 0.171. A proper fix needs to detect *alternating strength across bar lines* -- the signature of a metre read at the wrong multiple -- rather than scoring each candidate independently. The `two-thirds` preset is the honest workaround.

? **Section labels are guesses.** They come from energy and repetition, not from a model that knows what a chorus is.

? **Voice attribution is a proxy.** Band dominance plus percussiveness, not stem separation.

? **Render time.** About two minutes per twenty seconds at full size. Acceptable for a desktop tool, too slow for interactive scrubbing, which is why the preview is a single still frame.

? **`half-time` can over-segment.** On one track it produced eight figures from ten bars, which means the grouping learned nothing. Cohesion masks the symptom rather than fixing the cause.

? **No test suite in the conventional sense.** The guards are thorough about rendered output but there are no unit tests around the numeric internals, so a subtle regression inside `estimate_grid` would be caught only by its effect on the two reference tracks.

? **Two reference tracks.** Every threshold has been judged against a small sample. This is the single largest risk in the design.

20. Future enhancements

20.1 Analysis

? **Fix the metre multiple properly.** Score the *alternation* of downbeat strength across candidate bar lines. If every other detected bar line is strong and the ones between are weak, the metre is being read at the wrong multiple, and that is detectable directly rather than inferred.

? **Confidence, reported.** Every reading should carry a number and the page should say "fairly sure" or "this one is a guess, try two-thirds". A wrong answer presented with certainty is worse than an uncertain one presented honestly.

? **Show the alternatives.** Rather than a preset list, offer the top three readings the estimator actually considered, each with its bar length and score, and let one be chosen. This turns a menu of jargon into a menu of results.

? **Key and chord change detection**, for arrivals that land on harmonic movement rather than only on percussive events. A chorus often begins on a chord change a beat before the drums confirm it.

? **Vocal presence.** Even a crude "is someone singing here" band-energy heuristic would let the picture hold still under a vocal line and move between them, which is what a human editor does without thinking.

? **Optional heavier models when available.** If torch or librosa are present, use them and say so; fall back to the numpy path otherwise. The architecture already isolates listening behind `MusicAnalysis`.

? **Cache the analysis** keyed by file hash plus preset. Re-rendering the same track with a different style currently re-listens from scratch.

20.2 Choreography

- ? **Learn from what is kept.** If a render is exported rather than deleted, remember the reading and style that produced it and bias future defaults for that track.
- ? **Photo-aware casting.** Faces should not be wiped through, and a wide landscape wants a slower push than a close portrait. OpenCV can find faces without adding a dependency.
- ? **Real focal points.** The focal point is currently near the centre. Saliency, or a face, would make zooms land on the subject instead of the middle.
- ? **Photo ordering by content**, so consecutive photos differ in colour and composition rather than following file order.
- ? **Phrase awareness.** Four-bar and eight-bar phrases matter more than section labels; a build should intensify across the phrase, not step at its boundary.
- ? **A style that inverts.** Some music wants the picture to *stop* on the beat rather than change. Stillness on the accent is an effect the vocabulary cannot currently express.

20.3 Rendering

- ? **GPU or ffmpeg filter graph** for the affine work. The composition is simple enough to express as a filter chain, which would move the cost off Python entirely.
- ? **Progressive preview.** Render every tenth frame first for a rough animated preview, then fill in. Two minutes of waiting to see whether a choice was right is the biggest practical friction in the application.
- ? **Resolution presets** including 4K, and a draft mode at half size for fast iteration.
- ? **Text and titles** on the beat: lyrics, dates, place names.
- ? **Colour grading per section**, so a breakdown is cooler than a chorus.
- ? **Deterministic seeds surfaced**, so an edit can be reproduced exactly or nudged deliberately.

20.4 Application and interface

- ? **Compare two readings side by side.** Render twelve seconds under each and play both. This would have resolved the whole three-four problem in one look.
- ? **A style gallery.** Eight short clips, one per animation style, rendered once from the user's own photos and cached. Choosing from words is harder than choosing from pictures.
- ? **Timeline editing.** Show the grid, the figures and the clips, and allow a cut to be dragged or an arrival overridden per clip.
- ? **Undo across renders**, and keeping the last several outputs with their settings attached so a good one can be found again.
- ? **Batch mode:** one folder of photos against several tracks, or several styles, overnight.
- ? **Project files**, so an edit can be saved, reopened and shared.
- ? **Direct export presets** for the platforms people actually post to, with the correct dimensions and durations.
- ? **Packaging.** `electron-builder` into a signed installer, so the application does not depend on a hard-coded interpreter path in another project's virtual environment. This is the most obviously provisional thing in the current build.
- ? **Its own virtual environment.** Borrowing another project's is convenient and wrong.
- ? **Web and mobile**, which was always the intended direction. The engine is already a local HTTP service, so the interface is separable; the renderer is the part that would have to move server-side.

20.5 Verification

? **More reference tracks,** covering waltzes, swung rhythms, tracks with a slow introduction, live recordings with drift, and something in five-four. Every constant in this document has been judged against two songs.

? **Golden-frame comparison,** so a change to the renderer that alters output is noticed rather than discovered.

? **A timing check against hand-marked beats,** reporting the distribution of error rather than a pass or fail.

? **Unit tests for the numeric internals,** particularly `estimate_grid`, `condition` and the loop autocorrelation.

21. Cutting it down: making it simple enough

The application is too complex, and this section is the plan for fixing that. It is placed before the rebuilding notes deliberately, because if this work is done first, much of what precedes it becomes internal detail rather than something a person has to understand.

21.1 The diagnosis

The form accepts nineteen fields. The style list offers around twenty-four templates plus the automatic mode. There are eight reading presets, eight animation styles, four paces, three frame shapes, three fill modes, two piece shapes, and a beat-marker editor. Every one of them was added for a good reason, and together they are a wall.

The pattern behind the sprawl is consistent and worth naming:

Every uncertainty in the engine became a question for the user.

The engine could not reliably tell three-four from four-four, so `two-thirds` and `four-four` and `half-time` appeared. It could not tell how much variation was wanted, so `one-groove` and `fine-detail` appeared. It could not judge which arrival suited the photos, so eight animation styles appeared. Each preset is a confession, printed in the interface.

This is exactly backwards, because **the user has strictly less information than the engine does.** The engine has the spectrum, the grid, the figures and the loop strength. A person opening the application for the first time has a song they like and some photographs. Asking them to choose between "one-groove" and "fine-detail" asks them to arbitrate a question they have no way to answer, and then blames them for the result.

The correct posture is the opposite: **decide, show the result, and make rejection cheap.** A person cannot tell you that a bar is 3.25 seconds long. They can tell you instantly that one video looks better than another.

21.2 The target

Open the application. Drop photos. Drop a song. Press Make.

Three actions, no decisions. One visible choice, offered as pictures rather than words. One button that means "not that, try another". Measurable goals:

? from launch to a finished video: **three clicks and zero typed values**

- ? number of controls visible before the first render: ****at most four****
- ? time to something watchable: ****under thirty seconds****, not two minutes
- ? a first-time user should be unable to produce a bad result by choosing wrongly, because there should be nothing to choose wrongly

21.3 Choose the reading automatically

The eight reading presets exist because the estimator sometimes picks the wrong one. But the presets are not arbitrary: they are a small, enumerable set of candidate readings. The application can therefore ****run all of them and pick, instead of asking.****

Score each reading against an objective that describes what a good reading looks like, then keep the winner:

plausible bar length	1.6 to 4.0 seconds	strong preference
fewer figures	1 or 2 beats 8	strong preference
loop detected	and the stronger the better	
grid agreement	mean distance from grid beats to real onsets	
metre	four-four over three-four unless the evidence is clear	

On the reference track this objective picks the reading that is actually correct, which the sequential estimator does not:

reading	bar	figures	loop	verdict under the objective	
auto	1.63 s	2	yes	bar short, two figures	rejected
four-four	2.17 s	3	yes	three figures	rejected
half-time	4.33 s	8	no	no loop, eight figures	rejected
two-thirds	3.25 s	1	yes	plausible bar, one figure	CHOSEN <- correct

Cost is a few seconds of extra analysis, and it can be cut to near nothing by scoring candidate grids without recomputing the spectrogram, which is the expensive part and does not depend on the reading at all.

****This must be validated against more than two tracks before it is trusted.**** The objective above is a hypothesis fitted to a small sample. It is stated here as the plan, not as a finding. If it holds on twenty tracks, the reading dropdown disappears entirely and moves into the advanced drawer as an override.

21.4 Derive the pace instead of asking

"How busy" is arithmetic dressed up as taste. The user has already told the application everything it needs: how many photographs they chose, and how long the song is.

pace = (track length) / (photo count) snapped to the nearest musical division

Forty photos over a sixty-second track means one photo every 1.5 seconds, which is roughly a bar at most tempos. Eight photos over the same track means one every 7.5 seconds. Both are the obviously right answer, and neither needs a question. If every photo the user chose should appear, and it should, then pace is ****determined, not chosen.****

This deletes a control and also fixes a real complaint: a slow pace with many photos currently means most of the photographs are never shown at all.

21.5 Turn the animation style into pictures

Eight styles named "premium", "grids", "punch" and so on cannot be judged from their names. The descriptions help, but reading three lines of prose to understand a visual choice is the wrong medium.

Render **four two-second clips from the user's own photographs**, once, cached, and show them as a row of looping thumbnails. Four is enough: something building piece by piece, something soft, something with movement, something hard. The user points at the one they like.

This also lets the number of styles grow without making the interface worse, because the interface is a row of moving pictures rather than a list of words.

21.6 What to remove from view

Nothing here needs deleting from the engine. All of it moves behind a single **Advanced** disclosure that is closed by default and stays closed unless opened.

remove from the default path	why
????????????????????????????????	??
the 24 template styles	the automatic mode supersedes them; keep as "Legacy"
reading preset	chosen automatically (21.3), override in Advanced
pace / "how busy"	derived from photo count and length (21.4)
sensitivity	an analysis threshold; users cannot judge it
direction, intensity, cover_mode	template-era controls, meaningless in automatic mode
fill mode	implied by photo count once pace is derived
reveal_tiles, reveal_shape	the animation style already decides both
frame shape	inferred from the photos, with one toggle to override
the beat-marker editor	a repair tool; reachable from a result, not before one
output folder, name	defaulted and auto-named; shown after, not asked before
CapCut export	genuinely niche; a menu item, not a form field

Nineteen form fields become **three inputs and one visual choice**.

21.7 Replace configuration with rejection

The single most valuable addition is one button under a finished video:

[Not quite -- try another feel]

It re-renders with the next-best reading and a different style, keeping the photos and music. Press it three times and you have seen four interpretations without learning a single term.

This is why removing controls is safe. The presets existed as escape hatches from a wrong reading; this button is a better escape hatch, because it does not require knowing which hatch to take. Under the surface it walks exactly the same list of readings and styles that the dropdowns exposed -- it simply spends the user's attention on results instead of on vocabulary.

21.8 Make judging fast

Two minutes for twenty seconds of video is the hidden cause of the interface complexity. Because seeing a result is expensive, the interface tries to let people reason about settings in advance -- and reasoning in advance is precisely what a naive user cannot do.

Make the result cheap and the reasoning becomes unnecessary:

? ****Draft first.**** Render at half resolution, every second frame, no blur. Roughly eight times faster: a twenty-second draft in about fifteen seconds. Offer the full render as an explicit "Save in full quality".

? ****Render the middle first,**** where the chorus usually is, so the most representative part is watchable soonest.

? ****Keep the analysis cached**** by file hash, so trying another feel costs only the render.

21.9 Say less, and say it in plain words

The written account is thorough and almost none of it belongs on screen. Replace the panel of technical notes with one sentence, and keep the full account one click away:

before: designed from track.mp3: 111 BPM in 3/4, heard as intro, verse, chorus...
36 bars reduced to 2 repeating figure(s) at 12 slots per bar...
the figures were made to share one arrival (slide_in, from figure 0...)

after: A steady 74-beat groove that repeats every four bars, so every photo
arrives the same way. [why this?]

The detail is what makes the result arguable and must not be lost. It just should not be the first thing a person meets.

21.10 The order to do this in

Each step is useful alone, and each makes the next easier.

- | | | |
|---|---|-----------------------------------|
| 1 | derive the pace from photo count and length | deletes a control, fixes a bug |
| 2 | draft renders and cached analysis | makes everything else judgeable |
| 3 | "try another feel" | makes removal safe |
| 4 | automatic reading selection, validated on 20 tracks | deletes the biggest wall |
| 5 | style thumbnails from the user's own photos | replaces words with pictures |
| 6 | collapse the rest behind Advanced | 19 fields become 4 |
| 7 | one-screen layout: photos, music, Make | the shape the app should have had |

21.11 What must not be simplified away

Simplicity here means fewer decisions, not less care. Four things are load-bearing and must survive untouched:

? ****The full-view guarantee.**** `MIN_FULL_VIEW`` is why photographs are actually seen.

? ****Cohesion.**** It is the difference between an edit and a slideshow, and it must stay on by default without being asked about.

? ****The written account.**** One click away, not gone. It is the only way to argue with a result.

? ****The guards in section 16.**** A simpler interface over an unverified engine is worse than a complex interface over a verified one.

The engine's sophistication is not the problem. Exposing it is.

22. If rebuilding this from scratch

Six things are worth more than the rest of the document put together, because each was learned by getting it wrong first.

1. ****Decide tempo and metre together.**** Sequential estimation produces confident, wrong, self-consistent answers.
2. ****Detect the loop before comparing bars.**** On repetitive music, position in the loop beats neighbour comparison, because a slightly wrong grid makes identical bars look different.
3. ****Make the treatment a function of the music, not of a wish for variety.**** Anti-repetition logic produces exactly the restlessness it is meant to avoid.
4. ****Cohere the arrival by default, and do not gate it on a detector.**** When the detector is unsure, the right response is a calmer video, not a busier one.
5. ****Guarantee the full view.**** A minimum span where the photo is completely visible, enforced by shortening whatever overlaps it, is what separates an edit from a flicker.
6. ****Verify by measuring rendered frames.**** Every claim in section 16 is a number read off an actual output. A command exiting zero proves nothing.

And one about the environment: when a command appears to do nothing at all, suspect the shell before suspecting the code. Two separate multi-attempt dead ends in this project -- npm installing nothing, and checks apparently not running -- were both a shell refusing to execute rather than anything wrong with what was being executed.